

XS WALLET

Especificação Técnica

Carteira HD + Atomic Swaps - Aplicação Desktop

Documento	Especificação Técnica
Versão	0.2.0
Status	Fase 1 Completa - Boltz Client Production-Ready
Data	Janeiro 2026
Público-Alvo	Desenvolvedores, Arquitetos, Auditores
Backend	Go Core (xscore) com gRPC
Swap Provider	boltz-backend (self-hosted)

Índice

1. Sumário Executivo
2. Arquitetura do Sistema
 - 2.1 Visão Geral dos Componentes
 - 2.2 Modelo de Processos
 - 2.3 Fluxo de Dados
3. Especificação do Banco de Dados
 - 3.1 Design do Schema
 - 3.2 Definições de Tabelas
 - 3.3 Modelo de Concorrência
4. Protocolo de Atomic Swap
 - 4.1 Submarine Swap
 - 4.2 Reverse Swap
 - 4.3 Chain Swap
 - 4.4 Máquina de Estados
5. Especificações Criptográficas
6. Gerenciamento de Nodes
7. Modelo de Segurança
8. Especificações de API
9. Roadmap de Implementação

1. Sumário Executivo

XS Wallet é uma aplicação desktop self-custody que permite atomic swaps entre Bitcoin (BTC), Liquid (L-BTC) e Lightning Network (LN) usando Taproot e boltz-backend (self-hosted) como orquestrador de swap com nossa liquidez. O sistema mantém verdadeira auto-custódia enquanto fornece swaps cross-chain e cross-layer sem exigir que usuários operem sua própria infraestrutura de swap.

1.1 Capacidades Core

Capacidade	Implementação	Status
Go Core (xscore)	Backend autoritativo em Go com gRPC	Implementado
Boltz HTTP Client	Retry/backoff, Retry-After, erros tipados	Production-Ready
Boltz WebSocket	Single-writer, gap-recovery, connCtx	Production-Ready
Status Normalization	Table-driven por tipo de swap	Implementado
Swap Engine	CAS/optimistic locking, idempotência	Base Implementada
HD Wallet	Derivação BIP39/32/84/85	Parcial
Criptografia	Argon2id + AES-256-GCM	Projetado

Tabela 1.1: Matriz de Capacidades Core

1.2 Princípios de Design

Auto-Custódia: Usuário controla todas as chaves privadas; nenhum servidor detém fundos ou autoridade de assinatura.

Zero Trust: Todos os dados externos (respostas Boltz, dados de nodes) são verificados localmente antes do uso.

Crash Recovery: Todos os estados de swap são persistidos com operações idempotentes; swaps podem resumir após qualquer falha.

Restauração Determinística: Swaps pendentes podem ser restaurados apenas do mnemônico usando child seeds BIP85.

2. Arquitetura do Sistema

2.1 Visão Geral dos Componentes

O sistema é estruturado com Go Core (xscore) como backend autoritativo comunicando via gRPC com o frontend Electron. O swap orchestrator é o boltz-backend self-hosted com nossa liquidez. Esta arquitetura garante isolamento de falhas e operações de swap podem continuar mesmo se a UI reiniciar.

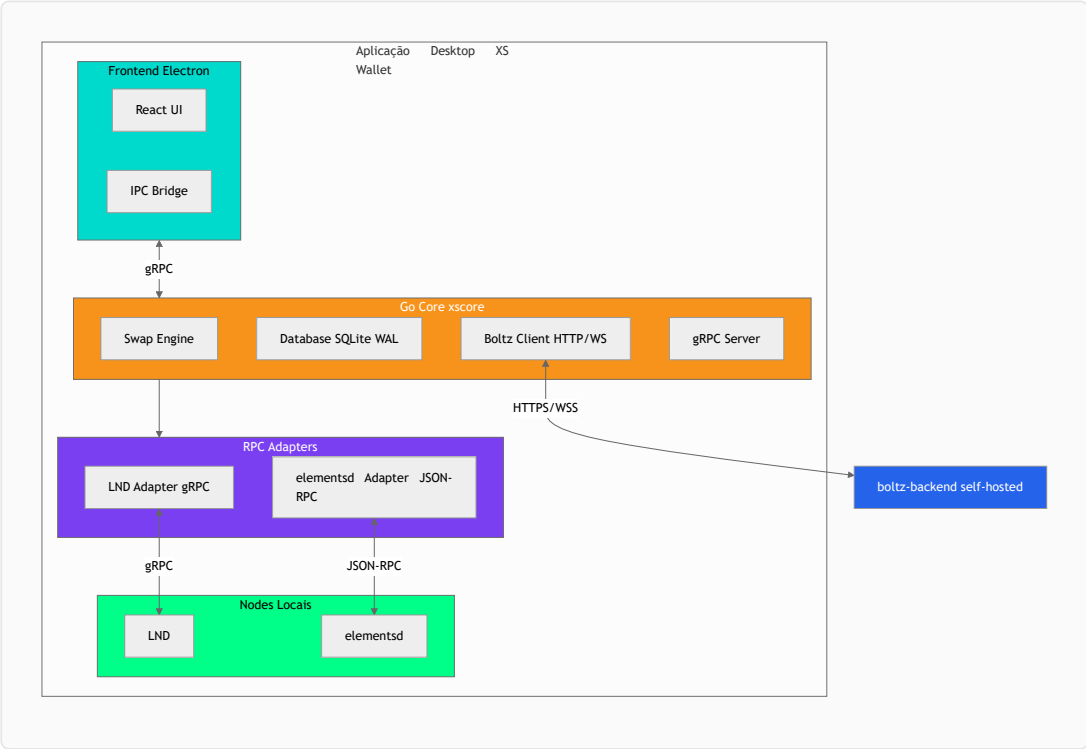


Figura 2.1: Diagrama de Arquitetura do Sistema

2.2 Modelo de Processos

Processo	Função	Benefício de Isolamento
Go Core (xscore)	Swap engine, database, adapters, Boltz client	Backend autoritativo isolado; crash-safe
Electron Frontend	React UI, comunicação gRPC com backend	UI isolada; pode reiniciar sem afetar swaps
boltz-backend	Orquestrador de swap self-hosted, nossa liquidez	Controle total; sem dependência de terceiros
LND	Lightning Network daemon (servidor gRPC)	Processo separado; autenticação macaroon
elements d	Liquid/Elements node (servidor JSON-RPC)	Processo separado; datadir próprio

Tabela 2.1: Modelo de Processos

2.3 Fluxo de Dados

Todas as operações iniciadas pelo usuário fluem através do gRPC para o Go Core. O backend mantém estado autoritativo no SQLite e coordena com chain adapters e boltz-backend (self-hosted) via REST/WS. **Invariante crítico: eventos WebSocket do boltz-backend são apenas triggers; estado on-chain/LND é sempre verificado antes de agir.**

3. Especificação do Banco de Dados

3.1 Design do Schema

O banco de dados usa SQLite com Write-Ahead Logging (WAL) para concorrência otimizada de leitura/escrita. Todas as mutações de estado de swap usam Compare-And-Swap (CAS) com locking otimista baseado em versão para prevenir race conditions sem bloquear queries.

3.1.1 Pragmas Críticos

Pragma	Valor	Justificativa
journal_mode	WAL	Leituras concorrentes durante escritas; crash recovery
synchronous	NORMAL	Balanço durabilidade/performance (seguro com WAL)
busy_timeout	5000	Espera 5s por locks; previne "database is locked"
foreign_keys	ON	Força integridade referencial
temp_store	MEMORY	Tabelas temporárias em RAM para performance
cache_size	-20000	Cache de ~20MB de páginas

Tabela 3.1: Pragmas SQLite

3.2 Definições de Tabelas

3.2.1 swaps - Estado Autoritativo do Swap

Coluna	Tipo	Constraints	Descrição
id	TEXT	PRIMARY KEY	Identificador único do swap (UUID)
kind	TEXT	CHECK(IN submarine,reverse,chain)	Tipo de swap
version	INTEGER	NOT NULL DEFAULT 0	Versão CAS para locking otimista
state	TEXT	CHECK(IN open,locked,...)	Estado atual da máquina de estados
locked_intent	TEXT (JSON)	NOT NULL quando state != open	Snapshot imutável de quote/fees
swap_key_index	INTEGER	NOT NULL	Índice de derivação BIP85
preimage_hash_hex	TEXT	CHECK(length=64)	Hash SHA256 da preimage R
musig_session_id	TEXT		Identificador de sessão MuSig2

Tabela 3.2: Tabela swaps (parcial - 40+ colunas total)

Invariante: Quando state não está em (open, failed, canceled), o campo locked_intent DEVE ser não-nulo. Isso é enforced via CHECK constraint e garante que todo swap ativo tenha um registro imutável dos parâmetros acordados.

3.2.2 Tabelas de Suporte

Tabela	Chave Primária	Propósito
--------	----------------	-----------

swap_events	seq (AUTOINCREMENT)	Log de auditoria imutável; permite replay/debug
swap_ops	(swap_id, op_key)	Ledger de operações idempotentes; previne broadcasts duplicados
utxo_reservations	(chain, txid, vout)	Previne double-spend de UTXO entre swaps
ln_reservations	payment_hash_hex	Previne pagamentos LN duplicados
app_config	key	Configuração do usuário; snapshot capturado em locked_intent

Tabela 3.3: Tabelas de Suporte

3.3 Modelo de Concorrência

Todas as transições de estado usam Compare-And-Swap (CAS) com transações atômicas:

```
-- Padrão de Update CAS
UPDATE swaps
SET state = 'locked',
    version = version + 1,
    updated_at = strftime('%Y-%m-%dT%H:%M:%fZ', 'now'),
    locked_intent = ?
WHERE id = ? AND version = ? AND state = 'open'
RETURNING version, state;

-- Se RETURNING está vazio: mismatch de versão (modificação concorrente)
-- Aplicação deve recarregar e tentar novamente ou abortar
```

Listagem 3.1: Padrão de Update CAS

4. Protocolo de Atomic Swap

4.1 Submarine Swap (On-chain → Lightning)

Submarine swaps convertem fundos on-chain (BTC ou L-BTC) para capacidade Lightning. O usuário bloqueia fundos em um HTLC P2TR; Boltz paga a invoice LN do usuário e clama o HTLC usando a preimage revelada.

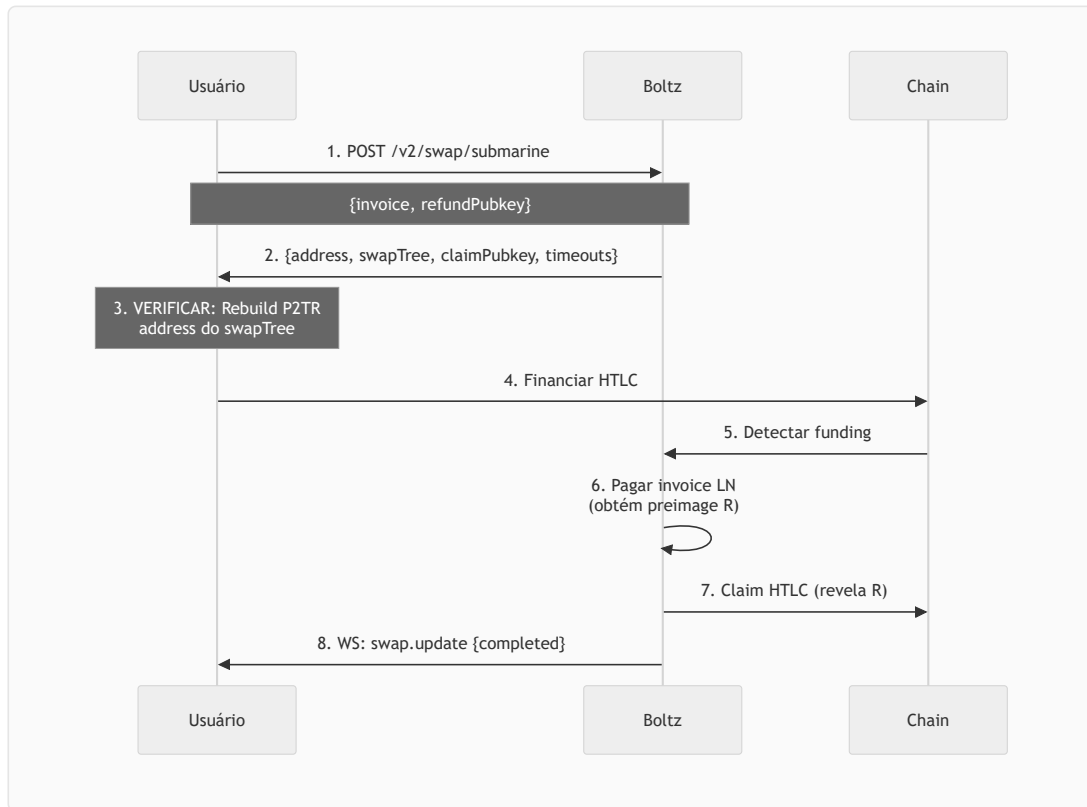


Figura 4.1: Sequência de Submarine Swap

4.2 Reverse Swap (Lightning → On-chain)

Reverse swaps convertem saldo Lightning para fundos on-chain. O usuário gera preimage R e hash $H = \text{SHA256}(R)$, paga uma hold invoice Boltz, e clama o HTLC on-chain revelando R.

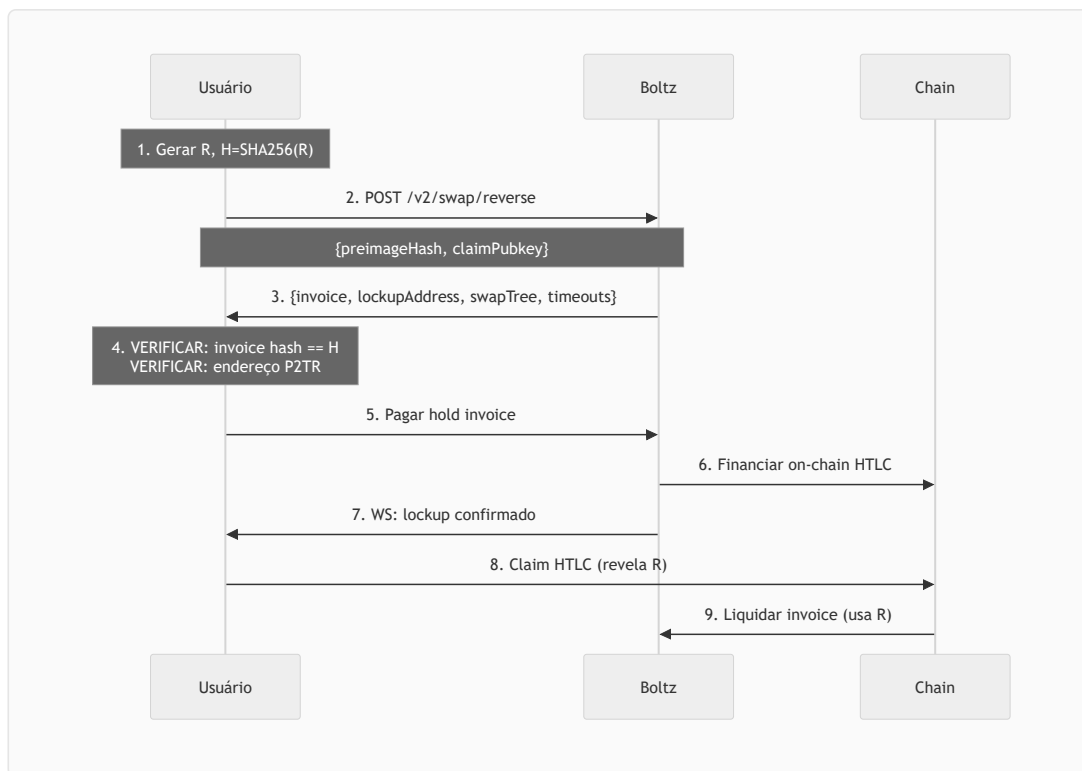


Figura 4.2: Sequência de Reverse Swap

4.3 Chain Swap (BTC ↔ Liquid)

Chain swaps são swaps atômicos cross-chain entre mainchain Bitcoin e sidechain Liquid. Timeouts são escalonados ($T_{\text{liquid}} < T_{\text{btc}}$) para prevenir race conditions.

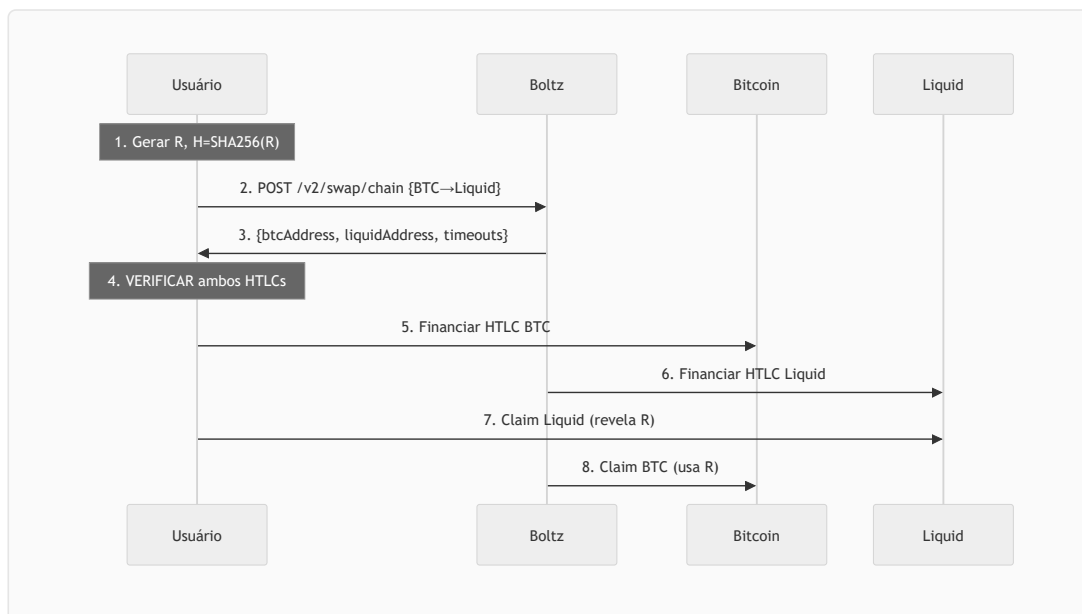


Figura 4.3: Sequência de Chain Swap

4.4 Máquina de Estados

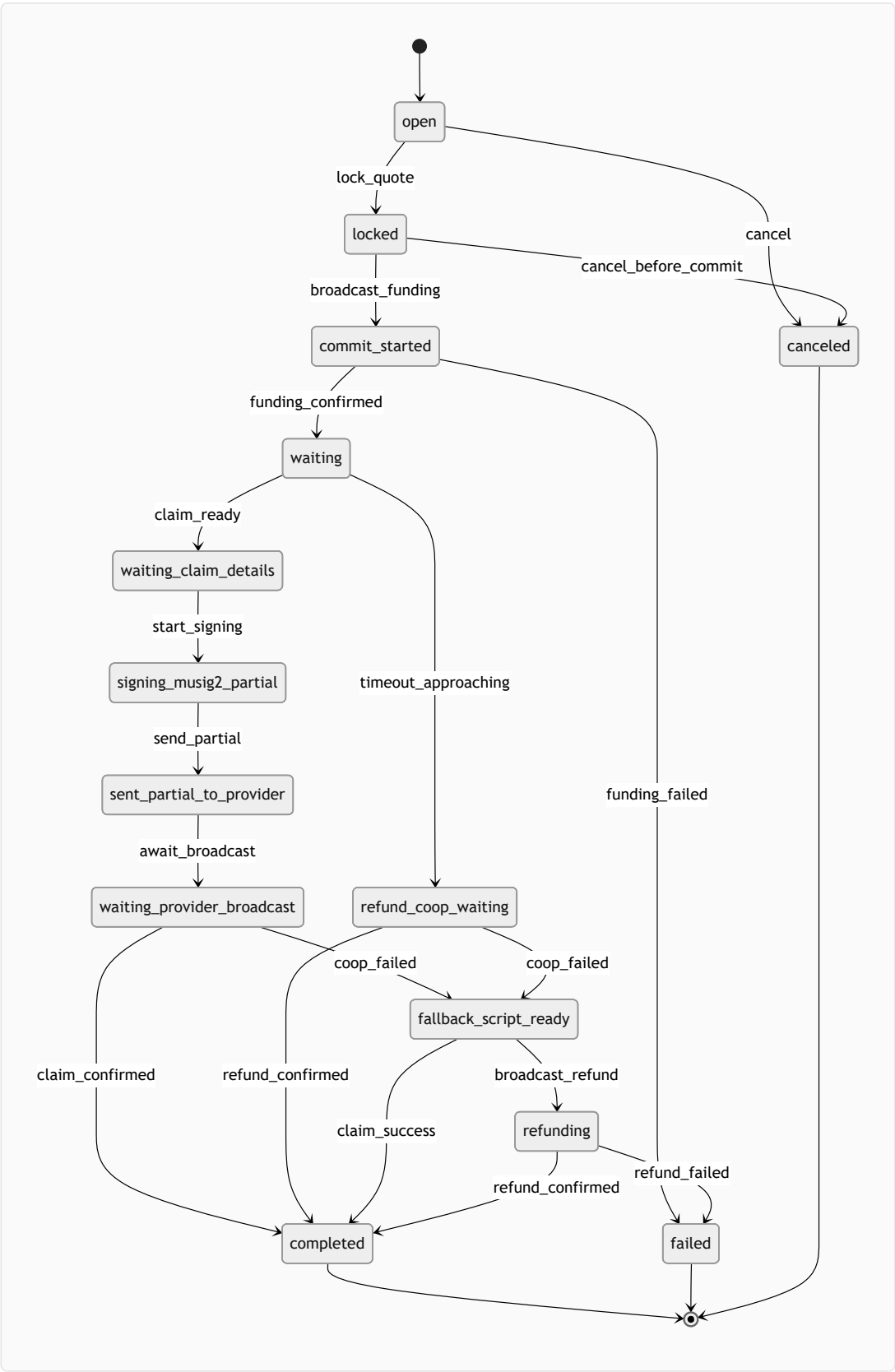


Figura 4.4: Máquina de Estados do Swap

Estado	Descrição	Transições
open	Estado inicial; quote aceito mas não bloqueado	locked canceled
locked	Parâmetros bloqueados; intent imutável	commit_started canceled
commit_started	Transação de funding broadcast	waiting failed

waiting	Aguardando ação da contraparte	waiting_claim_details refund_coop_waiting
signing_musig2_partial	Criando assinatura parcial MuSig2	sent_partial_to_provider
fallback_script_ready	Preparando claim/refund via script-path	refunding completed
completed	Estado terminal de sucesso	(terminal)
failed	Estado terminal de falha	(terminal)

Tabela 4.1: Máquina de Estados do Swap

5. Especificações Criptográficas

5.1 Hierarquia de Derivação de Chaves

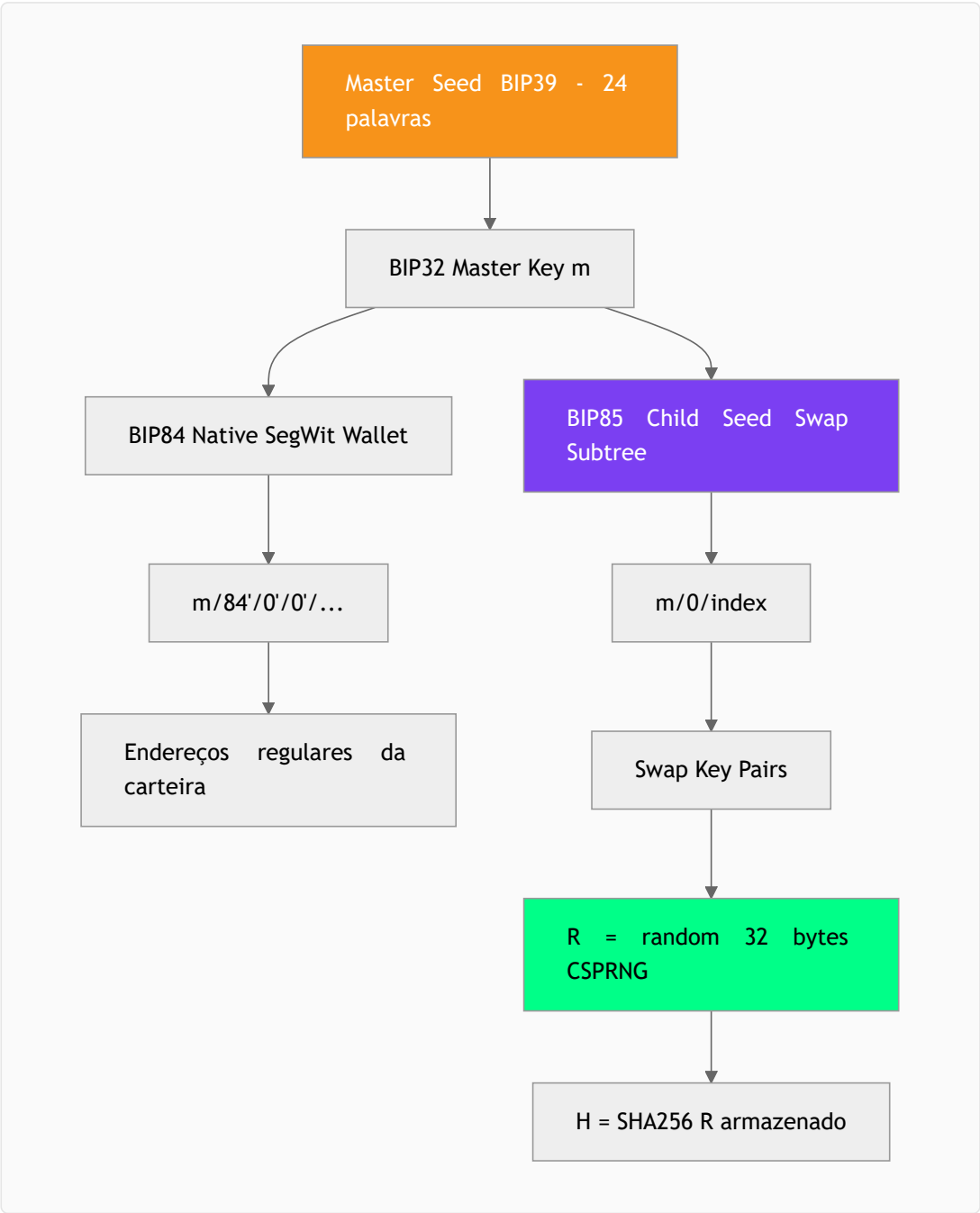


Figura 5.1: Hierarquia de Derivação de Chaves

5.1.1 Geração de Preimage (CRÍTICO)

REQUISITO DE SEGURANÇA: A preimage R **NÃO DEVE** ser derivada de chaves privadas. Derivar de privateKey cria acoplamento perigoso e risco de correlação se qualquer dado vazar.

Componente	Especificação
Geração de R	R = crypto.randomBytes(32) - CSPRNG do OS
Hash	H = SHA256(R) - enviado ao provider

Armazenamento	R criptografado no vault junto com swap state
Restauração	Via backup do banco de dados; R não é derivável do mnemônico
Ciclo de vida	R zerado da memória após claim confirmado

Tabela 5.0: Especificação de Preimage

Restauração de swaps pendentes: O mnemônico permite derivar chaves de assinatura; a preimage R é restaurada do backup do banco de dados criptografado.

5.2 Criptografia do Vault

Parâmetro	Valor	Justificativa
KDF	Argon2id	Memory-hard; resistente a ataques GPU/ASIC
Memória	64 MB	Balanço segurança/performance para desktop
Iterações	3	Recomendação padrão
Paralelismo	1	Derivação single-threaded
Salt Length	16 bytes	Random por carteira
Cipher	AES-256-GCM	Criptografia autenticada
IV Length	12 bytes	Padrão GCM
Tag Length	16 bytes	Tag de autenticação completa

Tabela 5.1: Parâmetros de Criptografia do Vault

Mitigação de Ataque Offline: Rate limiting (backoff exponencial) protege runtime mas não roubo offline do banco de dados. Para proteção aprimorada, derivar chave final de PIN + device secret armazenado no keychain do OS (Keychain/DPAPI/SecretService). MVP usa apenas PIN; device secret é v1.1.

5.3 Taproot + MuSig2 (Especificação Detalhada)

HTLCs de swap usam P2TR (Pay-to-Taproot) com protocolo MuSig2 para assinatura cooperativa eficiente:

5.3.1 Versão do Protocolo

Parâmetro	Valor
Protocolo Base	MuSig2 (BIP327 draft) - versão Boltz v2
Curva	secp256k1
Assinatura	BIP340 Schnorr
Número de Participantes	2 (usuário + provider)

Tabela 5.2a: Versão do Protocolo MuSig2

5.3.2 Agregação de Chaves

Componente	Especificação
Ordem de Agregação	Lexicográfica: sort([providerPubkey, userPubkey])
Key Aggregation	KeyAgg(L) conforme BIP327 §4.3
Tweak	Taproot tweak aplicado após agregação

Tabela 5.2b: Agregação de Chaves

5.3.3 Segurança de Nonce (CRÍTICO)

⚠ REUSO DE NONCE = PERDA DE FUNDOS: Um nonce reutilizado permite extração da chave privada.

Componente	Especificação
Geração de Nonce	<code>nonce = NonceGen(rand, sk, pk, msg, extra_in)</code> conforme BIP327 §4.1
Aux Randomness	<code>rand = crypto.randomBytes(32)</code> - OBRIGATÓRIO para cada sessão
Session Binding	Nonce vinculado a (swapId, sessionId, txHash) - impede reuso
Persistência	secnonce + pubnonce salvos em <code>musig_*</code> antes de enviar pubnonce
Verificação	Antes de assinar: verificar que secnonce corresponde ao pubnonce enviado
Cleanup	secnonce zerado após assinatura parcial gerada

Tabela 5.2c: Segurança de Nonce

5.3.4 Fluxo de Assinatura

```
1. KeyAgg: Q = KeyAgg([P_provider, P_user]) // ordem lexicográfica
2. NonceGen: (secnonce, pubnonce) = NonceGen(rand, sk, Q, msg, session_id)
3. NonceAgg: R = NonceAgg([pubnonce_provider, pubnonce_user])
4. Sign: psig = Sign(secnonce, sk, R, Q, msg)
5. Verify: PartialSigVerify(psig, pubnonce_user, P_user, R, Q, msg)
6. Aggregate: sig = PartialSigAgg([psig_provider, psig_user])
7. Verify: Verify(sig, Q, msg) // antes de broadcast
```

Listagem 5.1: Fluxo de Assinatura MuSig2

5.3.5 Fallback Script-Path

Condição	Script-Path Ação
Provider não responde (claim)	Claim via script-path com preimage R
Provider não coopera (refund)	Refund via script-path após timeout
Nonce/sessão corrompida	Abortar MuSig2, usar script-path

Tabela 5.2d: Fallback Script-Path

6. Gerenciamento de Nodes

6.1 Gerenciamento de Ciclo de Vida

O Node Manager lida com verificação de binários, spawning, monitoramento de health, e shutdown graceful de nodes embarcados. Cada node tem diretórios de dados isolados por rede.

Nota: A aplicação trata LND (gRPC) e elementsd (JSON-RPC) como RPCs primários; bitcoind, quando presente, é uma dependência do runtime e não uma integração primária do app.

Node	Versão	Diretório de Dados	Health Check	Papel
LND	0.17.3	%APPDATA%/xs-wallet/nodes/lnd/{network}/	getinfo (gRPC)	RPC primário (on-chain BTC + LN via LND)
elements	23.2.1	%APPDATA%/xs-wallet/nodes/liquid/{network}/	getblockchaininfo (JSON-RPC)	RPC primário (Liquid/Elements)
bitcoind (opcional)	26.0	%APPDATA%/xs-wallet/nodes/btc/{network}/	getblockchaininfo (RPC)	Dependência do runtime (quando necessário pelo stack; não é RPC primário do app)

Tabela 6.1: Especificações de Nodes

6.2 Verificação de Binários

Binários são baixados no primeiro uso de sources oficiais (GitHub Releases, bitcoincore.org) e verificados usando checksums SHA256 de um manifest assinado.

6.2.1 Especificação de Supply Chain

Componente	Especificação
Algoritmo de Assinatura	Ed25519 (manifest.sig)
Chave Pública	Pinned no binário da aplicação (src/constants/MANIFEST_PUBKEY)
Rotação de Chave	Nova chave anunciada via release N-1; aceita ambas por 2 releases
Manifest URL	https://github.com/xs-wallet/releases/latest/nodes-manifest.json
Fallback URLs	Espelhos: IPFS (ipfs://...) + backup CDN
Hash Algorithm	SHA256 (hex) para cada binário
Verificação	1) Verificar sig do manifest → 2) Verificar hash do binário → 3) Extrair

Tabela 6.2: Especificação de Supply Chain

Cadeia de Confiança: Sem servidor central de custódia ou execução de swaps. Apenas endpoints públicos de distribuição (GitHub Releases, IPFS). Usuário pode verificar assinatura manualmente via CLI.

7. Modelo de Segurança

7.1 Modelo de Ameaças

Ameaça	No Escopo	Mitigação
Roubo de seed (offline)	Sim	Argon2id + device secret (v1.1)
Brute force de PIN (runtime)	Sim	Backoff exponencial; lockout após 10 tentativas
Manipulação de swap	Sim	Verificação local de scripts; nunca confiar no provider
Ataque supply chain	Sim	Manifest assinado; verificação SHA256
Acesso físico ao dispositivo	Não (futuro)	Considerar suporte a hardware wallet
Comprometimento do OS (root)	Não	Fora de escopo; assume OS confiável

Tabela 7.1: Modelo de Ameaças

7.2 Verificação Criptográfica

Verificação	Quando	Ação em Falha
Rebuild endereço P2TR	Antes de financiar HTLC	Abortar swap; logar erro
Hash da invoice == hash da preimage	Antes de pagar hold invoice	Abortar swap; logar erro
Sanidade de timeout ($T_{liquid} < T_{btc}$)	Antes de bloquear	Abortar swap; logar erro
Assinatura parcial MuSig2	Antes de enviar ao provider	Abortar; tentar script-path
Confirmação on-chain	Após broadcast claim/refund	Retry com fee mais alta

Tabela 7.2: Pontos de Verificação

7.3 Modelo de Confiança do Provider (boltz-backend self-hosted)

Embora o sistema seja "self-custody", o provider (boltz-backend self-hosted) é um componente do stack e atua como orquestrador de swaps, não como custodiante. Esta seção define explicitamente o que o provider **pode** e **não pode** fazer:

7.3.1 O que o Provider NÃO PODE fazer

Ação	Motivo
Roubar fundos do usuário	HTLC exige preimage R (apenas usuário conhece em reverse swap) ou signature (apenas usuário possui chave)
Claim sem pagar	Submarine: preimage R só é revelada quando pagamento LN é recebido
Forjar scripts	Rebuild local do P2TR address; mismatch = swap abortado
Modificar termos após lock	locked_intent imutável no banco local

Tabela 7.3: Ações que o Provider NÃO PODE executar

7.3.2 O que o Provider PODE fazer

Ação	Impacto	Mitigação
------	---------	-----------

Censurar/recusar swaps	Denial of service	Liveness; usar outro provider
Atrasar resposta	Forçar fallback script-path	Timeout detectado; script-path claim/refund
Não cooperar no MuSig2	Key-path indisponível	Script-path fallback automático
Ver metadados do swap	Privacidade reduzida	Aceitável para MVP; Tor em v1.2
Cobrar fees altas	Custo elevado	Quote mostrado antes de lock; usuário decide

Tabela 7.4: Ações que o Provider PODE executar e mitigações

Invariante de Segurança: Em nenhum cenário o provider consegue roubar fundos. O pior caso é denial of service (forçar refund via script-path após timeout), resultando em perda de fees de mineração, não de fundos.

7.4 Electron Security Hardening

O processo Renderer (Chromium) é a superfície de ataque mais exposta. Medidas de hardening:

Configuração	Valor	Justificativa
contextIsolation	true	Isola preload script do contexto da página
nodeIntegration	false	Renderer não tem acesso direto a Node.js APIs
sandbox	true	Processo renderer em sandbox do Chromium
webSecurity	true	Enforce same-origin policy
allowRunningInsecureContent	false	Block mixed content

Tabela 7.5: Configurações de Hardening do Electron

7.4.1 IPC Security

Medida	Implementação
Whitelist de Canais	Apenas canais definidos em ALLOWED_CHANNELS são roteados
Validação de Payload	Zod schema validation em todos os payloads IPC
Rate Limiting	Máximo 100 mensagens/segundo por canal
Sanitização	DOMPurify para qualquer HTML renderizado

Tabela 7.6: Segurança IPC

7.5 Estratégia de Fees (RBF/CPFP)

Transações on-chain podem ficar stuck em mempool congestionado. Políticas de aceleração:

Situação	Ação	Trigger
Funding tx não confirma	RBF com fee estimation atualizada	Após 3 blocos sem confirmação
Claim tx não confirma	CPFP se necessário; RBF se possível	Após 2 blocos ou timeout < 12 blocos
timeout_approaching	Acelerar claim; preparar refund path	Timeout < 6 blocos
Refund tx não confirma	RBF agressivo (dobrar fee cada 2 blocos)	Timeout < 3 blocos

Tabela 7.7: Estratégia de Fees


```
// Fee estimation (RPC primários)
const estimateFee = async (chain: 'btc' | 'liquid', priority: 'low'|'medium'|'high') => {
  if (chain === 'btc') {
    // BTC on-chain via LND WalletKit (gRPC) – fonte primária para txs do wallet
    // Ex: EstimateFee / BumpFee (WalletKit) conforme políticas do daemon
    return await lndWalletKit.estimateFee({ priority });
  }

  // Liquid via elementsd JSON-RPC
  const feeRates = await elementsd.estimateSmartFee(priority === 'high' ? 1 : priority === 'medium' ? 3
  return Math.max(feeRates.feerate * 100000, MIN_RELAY_FEE);
};

// RBF/CPFP bump (alto nível)
const bumpFee = async (chain: 'btc'|'liquid', txid: string, newFeeRate: number) => {
  if (chain === 'btc') {
    // Bump via LND WalletKit quando aplicável (RBF/CPFP conforme política do daemon)
    return await lndWalletKit.bumpFee({ txid, newFeeRate });
  }
  // Liquid bump via elementsd (quando suportado pelo fluxo)
  return await elementsd.bumpfee(txid, newFeeRate);
};
```

Listagem 7.1: Estratégia de Fee Estimation e Bumping

8. Especificações de API

8.1 Swap Orchestrator API (boltz-backend self-hosted; compatível com Boltz v2)

Endpoint	Método	Propósito
/v2/swap/submarine	POST	Criar submarine swap (on-chain → LN)
/v2/swap/reverse	POST	Criar reverse swap (LN → on-chain)
/v2/swap/chain	POST	Criar chain swap (BTC ↔ Liquid)
/v2/swap/{id}	GET	Obter status do swap
/v2/swap/{id}/claim	POST	Submeter detalhes da transação de claim
/v2/swap/{id}/refund	POST	Submeter detalhes da transação de refund
/v2/ws	WebSocket	Atualizações de status em tempo real
/v2/nodes	GET	Obter routing hints para pagamentos LN

Tabela 8.1: Endpoints do boltz-backend (compatível com Boltz v2)

8.2 API IPC Interna

Canal	Direção	Payload
swap:create	Renderer → Backend	{kind, fromAsset, toAsset, amount}
swap:status	Backend → Renderer	{swapId, state, details}
wallet:balance	Renderer → Backend	{chain: btc liquid ln}
wallet:balance:response	Backend → Renderer	{confirmed, unconfirmed, pending}
node:status	Backend → Renderer	{elements: running stopped syncing, ind: running stopped syncing}

Tabela 8.2: Canais de API IPC

9. Roadmap de Implementação

Fase	Duração	Entregas	Status
Fundação	Completo	Schema, DB Layer, Node Manager, Decisões	✓ Concluído
Backend Core	2 semanas	Key Vault, Adapters, Boltz Client, Swap Engine	Próximo
Frontend	1.5 semanas	React UI, Onboarding, Dashboard, Interface de Swap	Pendente
Electron	1 semana	Processo Main, IPC, Auto-update, Deep Links	Pendente
Packaging	0.5 semanas	Instaladores, Code Signing, Testes E2E	Pendente

Tabela 9.1: Roadmap de Implementação

9.1 Roadmap Pós-MVP

Versão	Target	Features
v1.1	Q2 2026	Suporte a hardware wallet, SQLCipher, device secret
v1.2	Q3 2026	Integração Coinjoin, Payjoin, LNURL
v2.0	Q4 2026	Assets RGB, Taproot Assets, suporte DLC

Tabela 9.2: Roadmap Pós-MVP

XS Wallet - Especificação Técnica v0.1.0
Janeiro 2026 • Fundação Completa
Self-Custody • Zero Trust • Don't Trust, Verify